

## Process

**process** : This object provides information about, and control over, the current Node.js process.

**process.argv** : returns an array containing the command-line arguments passed when the Node.js process was launched.

```
process.cwd() ; to see the current working directory
```

`process.argv` is a Node.js property that returns an array containing the command-line arguments passed to the program.

```
</> JavaScript
console.log(process.argv);
```

## Example

Suppose your file is `app.js` and you run:

```
node app.js hello 123 abc
```

Then:

```
</> JavaScript
console.log(process.argv);
```

Output:

```
</> JavaScript

[
  'C:\\Program Files\\nodejs\\node.exe', // Path to Node executable
  'C:\\Users\\YourName\\app.js',        // Path to the current file
  'hello',                               // First user argument
  '123',                                 // Second user argument
  'abc'                                  // Third user argument
]
```



## Meaning of each index

| Index                               | Value                                      |
|-------------------------------------|--------------------------------------------|
| <code>process.argv[0]</code>        | Path to the Node.js executable             |
| <code>process.argv[1]</code>        | Path to the JavaScript file being executed |
| <code>process.argv[2]</code> onward | Arguments provided by the user             |

```
PS C:\Users\kumar\OneDrive\Desktop\notess\notess\backend> node script.js
[
  'C:\\Program Files\\nodejs\\node.exe',
  'C:\\Users\\kumar\\OneDrive\\Desktop\\notess\\notess\\backend\\script.js'
]
PS C:\Users\kumar\OneDrive\Desktop\notess\notess\backend> node script.js hello utsav
[
  'C:\\Program Files\\nodejs\\node.exe',
  'C:\\Users\\kumar\\OneDrive\\Desktop\\notess\\notess\\backend\\script.js',
  'hello',
  'utsav'
]
```

```
let args = process.argv ;

for(let i = 2 ; i < args.length ; i++) {
  |   console.log("Hello to " , args[i]) ;
}
```

Input

Output:

```
PS C:\Users\kumar\OneDrive\Desktop\notess\notess\backend> node script.js utsav ram modi rahul gandhi
Hello to  utsav
Hello to  ram
Hello to  modi
Hello to  rahul
Hello to  gandhi
```

## module.exports

*requiring files*

**require()** a built-in function to include external modules that exist in separate files.

**module.exports** a special object

JS script.js

JS math.js M X

backend > JS math.js > ...

```
1  const sum = (a , b) => a + b ;
2  const mul = (a , b) => a * b ;
3  let a = 5 ;
4  let b = 3 ;
5  const g = 9.8 ;
6  const PI = 3.14 ;
7
8  module.exports = {
9      sum : sum ,
10     mul : mul ,
11     g : g ,
12     PI : PI ,
13 } ;
14
15 // module.exports = 123;
```

JS script.js X

JS math.js M

backend > JS script.js > ...

```
1  // let n = 5 ;
2  // for(let i = 0 ; i < n ; i++) {
3  //      console.log("hello" , i) ;
4  // }
5  // console.log("bye!") ;
6
7  // console.log(process.argv) ;
8
9  // let args = process.argv ;
10
11 // for(let i = 2 ; i < args.length ; i++) {
12 //      console.log("Hello to " , args[i]) ;
13 // }
14
15 // access property of another file
16
17 const someValue = require("./math") ;
18 console.log(someValue) ; |
```

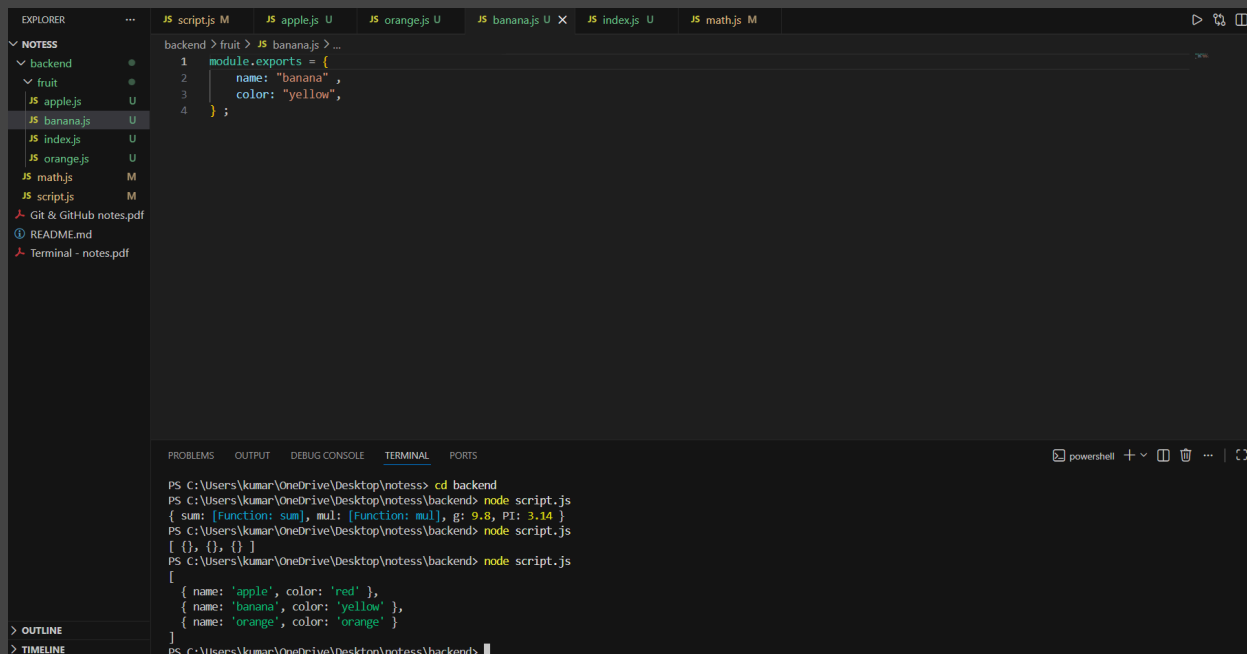
```
PS C:\Users\kumar\OneDrive\Desktop\notess\backend> node script.js
{ sum: [Function: sum], mul: [Function: mul], g: 9.8, PI: 3.14 }
PS C:\Users\kumar\OneDrive\Desktop\notess\backend> |
```

# module.exports

requiring directories

**require()** a built-in function to include external modules that exist in separate files.

**module.exports** a special object



The screenshot shows a VS Code editor with a file explorer on the left containing a project named 'NOTES' with subfolders 'backend', 'fruit', 'apple.js', 'banana.js', 'index.js', 'orange.js', 'math.js', and 'script.js'. The main editor displays the content of 'banana.js':

```
backend > fruit > JS banana.js > ...
1 module.exports = {
2   name: "banana",
3   color: "yellow",
4 };
```

The terminal at the bottom shows the execution of a script that uses these modules:

```
PS C:\Users\kumar\OneDrive\Desktop\notess> cd backend
PS C:\Users\kumar\OneDrive\Desktop\notess\backend> node script.js
{ sum: [Function: sum], mul: [Function: mul], g: 9.8, PI: 3.14 }
PS C:\Users\kumar\OneDrive\Desktop\notess\backend> node script.js
[ {}, {}, {} ]
PS C:\Users\kumar\OneDrive\Desktop\notess\backend> node script.js
[
  { name: 'apple', color: 'red' },
  { name: 'banana', color: 'yellow' },
  { name: 'orange', color: 'orange' }
]
```

## NPM (Node Package Manager)

npm is the standard package manager for Node.js.

① library of packages

code

# Installing Packages

node\_modules The node\_modules folder contains every installed dependency for your project.

**package-lock.json** It records the exact version of every installed dependency, including its sub-dependencies and their versions.

| Task                  | Command Prompt (cmd)       | PowerShell / Git Bash                                                              |  |
|-----------------------|----------------------------|------------------------------------------------------------------------------------|--|
| List files            | <code>dir</code>           | <code>ls</code>                                                                    |  |
| Change directory      | <code>cd folderName</code> | <code>cd folderName</code>                                                         |  |
| Go back               | <code>cd ..</code>         | <code>cd ..</code>                                                                 |  |
| Clear screen          | <code>cls</code>           | <code>clear</code> (or <code>cls</code> in PowerShell)                             |  |
| So in your case, run: |                            |  |  |

## package.json

The package.json file contains descriptive and functional **metadata** about a project, such as a name, version, and dependencies

*npm init*

```
1 import figlet from "figlet";
2
3 figlet.text(
4   "Utsav ",
5   {
6     font: "Ghost",
7     horizontalLayout: "default",
8     verticalLayout: "default",
9     width: 80,
10    whitespaceBreak: true,
11  },
12  function (err, data) {
13    if (err) {
14      console.log("Something went wrong...");
15      console.dir(err);
16      return;
17    }
18    console.log(data);
19  }
20 );
21
```

## require v/s import

`import { sum } from "./math.js"`

We can't selectively load only the pieces we need with require but with import, we can selectively load only the pieces we need, which can save memory.

Loading is synchronous for 'require' but can be asynchronous for 'import'.